

Backend Engineer Assignment

Backend Engineer — Take-Home Assignment

Objective

Assess your ability to design and build backend systems that ingest, store, and query high-frequency sensor/telemetry data — plus your ability to reason about scaling such a system.

Scenario

Assume you're building the backend for a system that receives motion data from smartphones at high frequency (10–50 events per second per device): GPS, accelerometer, and gyroscope readings. Your job is to build a minimal but correct ingestion + query service, and describe how you'd scale it.

Part 1: Data Ingestion API (Coding)

Build a REST API with the following endpoint:

```
POST /api/v1/telemetry/ingest
```

Request body: a **batch** of events (arrays, not single events — mobile devices batch data and upload periodically). Example payload:

```
{
  "device_id": "DVC-1029",
  "events": [
    {
      "timestamp": 1730894521123,
      "lat": 12.9716,
      "lon": 77.5946,
      "speed_kmph": 42.5,
      "accel_x": 0.12,
      "accel_y": -0.03,
      "accel_z": 9.81,
      "gyro_x": 0.002,
      "gyro_y": -0.001,
      "gyro_z": 0.0
    },
    {
      "timestamp": 1730894521173,
      "lat": 12.9716,
      "lon": 77.5947,
      "speed_kmph": 42.8,
      "accel_x": 0.14,
      "accel_y": -0.02,
      "accel_z": 9.80,
      "gyro_x": 0.001,
      "gyro_y": -0.001,
      "gyro_z": 0.0
    }
  ]
}
```

Field definitions:

Top-level:

Field	Type	Notes
device_id	string	unique per device
events	array	1 to 500 event objects

Per event (all fields required):

Field	Type	Notes
timestamp	int (epoch millis)	event time, not server receipt time
lat, lon	float	GPS coordinates
speed_kmph	float	instantaneous speed
accel_x / accel_y / accel_z	float	accelerometer, m/s ²
gyro_x / gyro_y / gyro_z	float	gyroscope, rad/s

Validation rules (an event failing any of these should be rejected, not stored):

Rule	Condition
Required fields	All per-event fields listed above must be present
timestamp	Positive integer; not more than 24 hours in the future relative to server time
lat	Between -90 and 90
lon	Between -180 and 180
speed_kmph	Between 0 and 300
accel / gyro fields	Must be numeric (any value)

Request-level failures — a missing or empty `device_id`, a missing or empty `events` array, or a batch larger than 500 events — should reject the whole request with `400` rather than being reported per-event.

Requirements: - Validate each event independently — **partial success is expected**. If 8 of 10 events are valid, store those 8 and report the other 2 as rejected. Do not fail the entire batch. - Handle duplicate events gracefully — the same `device_id + timestamp` may arrive twice due to network retries. A duplicate should not create a second stored row, and should be reported separately from rejections (see response format below). Duplicates may appear **within the same batch** as well as across separate requests — handle both. - Store data so it supports efficient time-range queries per device

Expected response (HTTP `200`):

```
{
  "device_id": "DVC-1029",
  "accepted": 8,
  "duplicates": 1,
  "rejected": 1,
  "errors": [
    { "index": 4, "reason": "speed_kmph out of range" }
  ]
}
```

`index` refers to the event's position in the submitted `events` array.

Part 2: Query API (Coding)

Build:

```
GET /api/v1/telemetry/summary?device_id=DVC-1029&from=<epoch_ms>&to=<epoch_ms>
```

Both `from` and `to` are **inclusive** bounds, in epoch milliseconds. Return, for that device within the window:

- Average speed, in kmph
- Maximum acceleration magnitude, computed per event as $\sqrt{(\text{accel_x}^2 + \text{accel_y}^2 + \text{accel_z}^2)}$, then the maximum across all events in the window
- Total event count

Expected response (HTTP `200`):

```
{
  "device_id": "DVC-1029",
  "from": 1730894520000,
  "to": 1730894580000,
  "avg_speed_kmph": 42.65,
  "max_accel_magnitude": 9.81,
  "event_count": 2
}
```

Edge cases to handle: - If no events exist in the range, return 200 with `event_count: 0` and `null` for the two aggregate values - If `from` is greater than `to`, or a required query parameter is missing, return 400 with a clear error message

Part 3: System Design (Write-up — no code required)

In a short document (max 1–2 pages), answer the following. **Give concrete numbers and named technologies — answers like “use a scalable queue” or “pick a suitable database” will not score well.**

1. **Throughput and storage.** 10,000 devices each send a batch every ~2 seconds, at the 10–50 Hz sampling rate from the scenario. Calculate the resulting requests/second and events/second, and show your working. Then estimate the storage growth per day, stating the assumptions behind your per-event size. Based on those numbers, name the specific database you would use and the partitioning/sharding key you would choose, and say what breaks first if you don't partition.
2. **Failure and load behaviour.** For each of these, describe what your system actually does — not just the general principle:
 - An event arrives 45 minutes late, after a summary for that window has already been served
 - The same batch is retried three times because the device never received the response
 - Traffic increases 10x within one minute
3. **AWS implementation.** Map your architecture to specific AWS services across ingestion, buffering, compute, and storage. For each, give one sentence on why that service over the obvious alternative (for example, Kinesis vs SQS, or Timestream vs DynamoDB vs RDS). No deployment required.

What You Can Assume

- No authentication/authorization is required for this assignment
- A single-region, single-node deployment is fine for the coding part — the design write-up should reason about scale separately
- Any language, framework, and database of your choice is fine
- In-memory storage, SQLite, Postgres, or any other DB is acceptable for the coding part; your Part 3 answer doesn't need to match your code's actual storage choice
- A sample/mock data generator is welcome but not required

Deliverables

1. Source code (GitHub link or zip) with a README covering setup and how to run it
2. A few sample `curl` or Postman requests showing both endpoints working
3. The Part 3 system design write-up as a separate `.md` or `.pdf` file

Your README must also include a short “Design Notes” section (roughly half a page) covering: - Your table/collection schema and which indexes you created, and why - How you enforce duplicate detection, and what happens under concurrent requests for the same device - One thing you would do differently if this were going to production

This section carries real weight in evaluation — a working submission with no reasoning behind it scores lower than a slightly rougher one that explains its choices.

Submit to: `[add your email / form link here]`

Time Expectation

Designed to take **4–6 focused hours**. Please submit within **3 days** of receiving this. We're not evaluating speed — a clean, well-reasoned submission matters more than covering every possible edge case.

Evaluation Criteria

- API design and correctness
- Data validation and error handling
- Code quality, structure, and readability
- Schema and indexing choices, and the reasoning given for them
- Handling of duplicates, concurrency, and edge cases
- Whether the system design answers use concrete numbers and named technologies, not generic principles
- Understanding of time-series / high-frequency data challenges
- Familiarity with relevant AWS services and justification for choices over alternatives

Bonus (Optional — not required)

- Basic tests for the ingestion endpoint
- Rate limiting or backpressure handling
- A Dockerfile for easy setup